COMMUNITY SUMMIT

Microsoft Fabric



October 11–15, 2026
Gaylord Opryland Resort

The largest independent Microsoft AI & Business Applications User Conference on the planet.

COMMUNITY SUMMIT

CI/CD for Power BI and Fabric

Git, TMDL, and deployment pipelines you can actually implement



@ 2026 Dynamic Communities

Speaker



Michael McKinley
BI Development Pillar Lead,
White Cap Supply

Michael McKinley is a data analytics consultant, instructor, and Power BI evangelist with a passion for turning complexity into clarity. At White Cap Supply, he leads data intelligence efforts that empower senior leaders with actionable insights.

Outside the office, Michael coleads the Nashville Power BI User Group, teaches live training courses, and speaks at conferences to help others harness the full potential of Power BI, the Power Platform, and Microsoft Fabric.

WATCH ON YOUTUBE

The Data Practitioner

@thedatapractitioner

@ 2026 Dynamic Communities

Learning Objectives

1. Design a practical CI/CD pipeline for Power BI and Fabric that incorporates branching strategies, deployment pipelines, and appropriate governance guardrails for your team's environment.

2. Explain how Git integration and TMDL work together to transform Power BI and Fabric assets into versionable, auditable code that supports change tracking, collaboration, and compliance requirements.

3. Evaluate where Terraform fits into a Git-based deployment framework, understanding how it extends your pipeline to automate workspace provisioning and Fabric object management at scale.

@ 2026 Dynamic Communities

Session Agenda

1. **Two problems that lead to CI/CD**
   Traceability and auditability
2. **Right-size your approach**
   From doing nothing to Git + CI/CD
3. **Power BI is code now**
   PBIP, TMDL, and Git
4. **Branching and CI guardrails**
   Pull request with build validation
5. **Promoting across environments**
   Deployment pipelines
6. **The audit trail**
   Mapping it to change controls
7. **Where Terraform fits**
   Platform vs. content

@ 2026 Dynamic Communities

Two problems that lead to CI/CD

COMMUNITY SUMMIT

SHOWS UP CONSTANTLY

What changed, who changed it, and why?

A number moves or a visual breaks. The Service tells you part of the story, and for plenty of teams that's enough. When it isn't, you need Git.

RARER, BUT HEAVIER

Can you prove it to an auditor?

Under Sarbanes-Oxley or similar controls, the auditability and traceability of BI assets isn't optional. A Git-integrated deployment process becomes a requirement.

Show of hands: *who has chased a changed number without knowing what changed?*

@ 2026 Dynamic Communities

What Git adds to the Service

	Power BI Service alone	With Git + CI/CD
What changed?	A new version was published	A line-by-line diff of every measure, table, and visual
Who changed it?	Who published it (activity log)	The author of every commit
Why?	Ask around	The commit message and linked work item
Was it reviewed?	No built-in review gate	A pull request with required reviewers
Can we undo it?	Republish an old copy, if you kept one	Revert the exact commit

@ 2026 Dynamic Communities

Right-size your approach

COMMUNITY SUMMIT

TODAY'S FOCUS

STEP 1
Do nothing
Free and nimble, with no protection, recovery, or traceability

STEP 2
File versioning
Copies on a network share. Easy, but manual and easy to forget

STEP 3
OneDrive / SharePoint
A version on every save. Hard for teams editing the same file

STEP 4
Deployment pipelines
Dev → Test → Prod with compare. Moves whole items, not single changes

STEP 5
Git + CI/CD
Diff, author, and reason for every change, with automated checks

More effort and cost → more capability and assurance

@ 2026 Dynamic Communities

Power BI is code now

COMMUNITY SUMMIT

BEFORE

Sales.pbix

One binary file. No diff, no merge, no review.

NOW

PBIP, TMDL, and PBIR are generally available, and PBIR is now the default report format.

Every table, measure, and visual is a text file you can diff, review, and merge.

Fabric Git integration writes the same folder structure to your repo.

```
Sales project folder

Sales.pbip
Sales.Report/
  definition.pbir
  definition/
    report.json
    pages/
      Overview/
        page.json
        visuals/…
Sales.SemanticModel/
  definition.pbism
  definition/
    model.tmdl
    relationships.tmdl
    tables/
      Sales.tmdl
      Date.tmdl
```

@ 2026 Dynamic Communities

A measure change, as Git sees it

COMMUNITY SUMMIT

```
Sales.SemanticModel/definition/tables/Sales.tmdl

table Sales

    measure 'Gross Margin %' =
-           DIVIDE ( [Gross Margin], [Total Sales] )
+           DIVIDE ( [Gross Margin], [Net Sales] )
        formatString: 0.0%
        displayFolder: Profitability
```

`3f9c2ab`

Michael McKinley

#1423 Use net sales as the margin denominator

Pull request #57 · approved by the model owner

WHAT

The diff, line by line

WHO

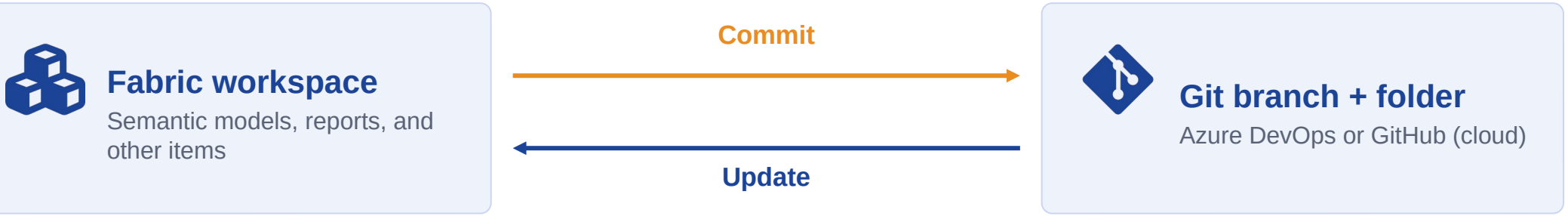
The commit author and the PR reviewer

WHY

The commit message and linked work item

@ 2026 Dynamic Communities

Fabric Git integration in one slide



Connect

Each workspace syncs with one branch and folder in Azure DevOps or GitHub.

Commit and update

Changes move in both directions through the workspace's Source control pane.

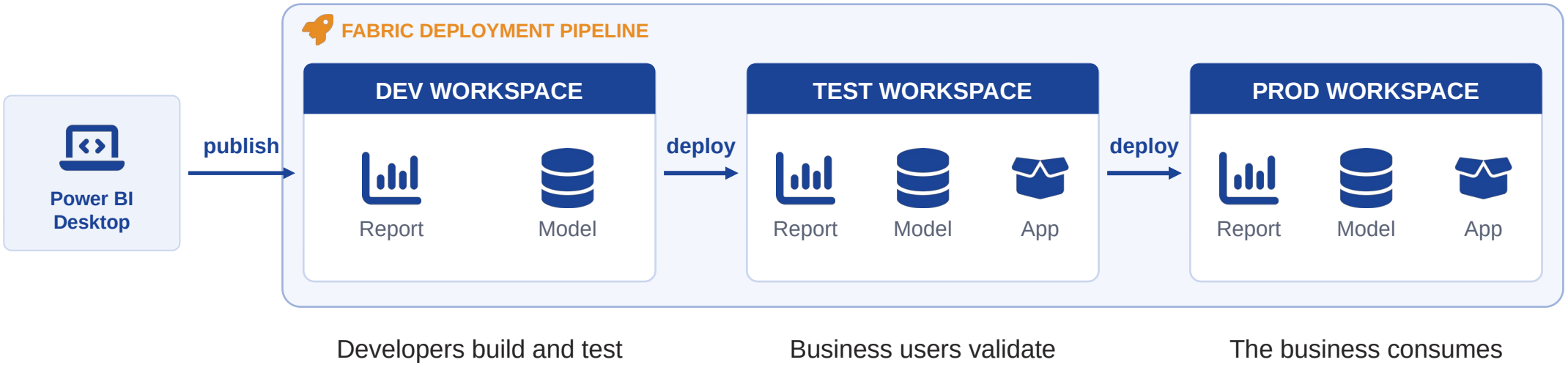
Branch out

Create a feature workspace on a new branch, work in isolation, then merge back.

@ 2026 Dynamic Communities

Option 1: In-Service pipelines only

Fabric deployment pipelines promote content from Dev to Test to Prod with no external Git. The simplest path to multiple environments.



Developers build and test

Business users validate

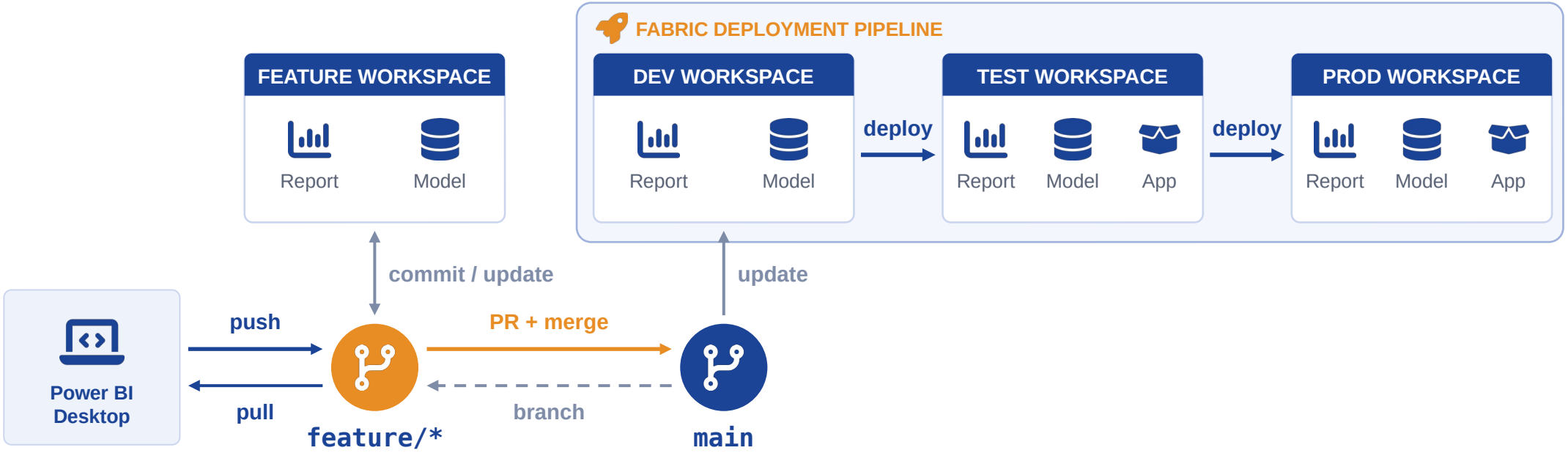
The business consumes

Best for: Teams new to version control that want a promotion workflow without Git complexity.

@ 2026 Dynamic Communities

Option 2: Feature branches + pipelines

Developers work in feature branches, peer-review through pull requests, and merge to main. Main syncs to the Dev workspace; deployment pipelines promote to Test and Prod.

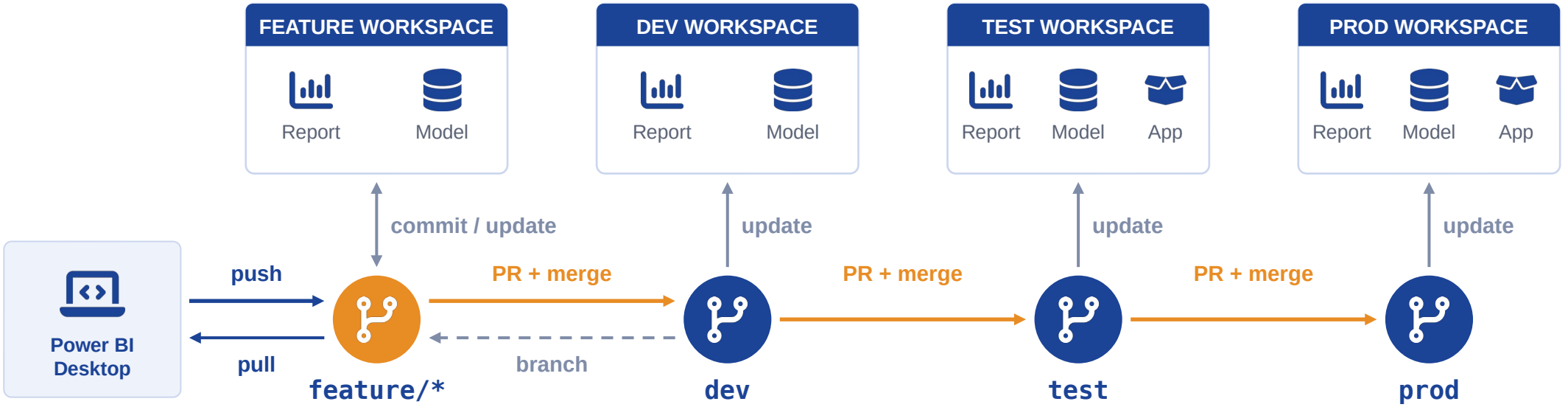


Best for: Teams with Git skills that need parallel development, code review, and granular change tracking.

@ 2026 Dynamic Communities

Option 3: Three permanent branches

Permanent Dev, Test, and Prod branches mirror your environments. Changes are promoted by merging between branches, manually or through CI/CD.



Best for: Teams that want clear environment separation with explicit promotion between stages.

@ 2026 Dynamic Communities

Guardrails on the main branch

Azure Repos branch policies

- Require a pull request: no direct pushes to main
- At least one reviewer, and authors can’t approve their own changes
- A linked work item is required
- All comments resolved
- Build validation must pass

What the build checks

Semantic models
Best Practice Analyzer rules (Tabular Editor)

Reports
PBI Inspector rules

Your standards
A team rules file kept in the repo

Optional
DAX query tests against known values

Tabular Editor and PBI Inspector are free community tools.

@ 2026 Dynamic Communities

Build validation in Azure Pipelines

```
pipelines/pr-validation.yml (condensed; full version in the repo)

trigger: none   # run by the branch policy on PRs to main
pool:
  vmImage: windows-latest

steps:
- checkout: self
- pwsh: |
    # Tabular Editor 2 + rules downloaded in a prior step
    $te    = "$(Agent.TempDirectory)\TE\TabularEditor.exe"
    $rules = "rules\Rules-Dataset.json"
    Get-ChildItem -Recurse -Directory -Filter *.SemanticModel |
      ForEach-Object {
        & $te "$($_.FullName)\definition" -A $rules -V
      }
  displayName: Best Practice Analyzer
```

`-A rules.json`

Runs Best Practice Analyzer with your team's rules

`-V`

Writes results into the PR as Azure DevOps errors and warnings

`Severity 3`

Fails the build and blocks the merge. Severity 2 is a warning.

@ 2026 Dynamic Communities

Code-first deployment with fabric-cicd

fabric-cicd (Python)

MICROSOFT'S OPEN-SOURCE LIBRARY · CODE-FIRST, FROM THE REPO

Deploys Fabric items straight from your Git repo to any workspace, using the Fabric REST APIs. The same script runs on your laptop and in a pipeline.

Deploys from Git

Publishes semantic models, reports, and other Fabric items straight from the repo.

One repo, every environment

parameter.yml swaps workspace IDs, endpoints, and connection details for dev, test, and prod.

Handles the ordering

Publishes models before the reports that depend on them, and can clean up orphaned items.

Runs anywhere Python runs

Interactive sign-in on your laptop; a service principal in Azure Pipelines or GitHub Actions.

Choose it when: every deployment should come from Git, repeat across many workspaces, and get reviewed like code.

You'll need: a service principal with Contributor on the target workspaces, and the tenant setting that lets service principals use Fabric APIs.

@ 2026 Dynamic Communities

fabric-cicd in practice

deploy/deploy.py (condensed)

```python
from azure.identity import AzureCliCredential
from fabric_cicd import FabricWorkspace, publish_all_items

workspace = FabricWorkspace(
    workspace_name="Sales [Prod]",
    environment="prod",
    repository_directory="src",
    item_type_in_scope=["SemanticModel", "Report"],
    token_credential=AzureCliCredential(),
)
publish_all_items(workspace)
```

First deployment of a model with data sources? Set its credentials once in Fabric; later deployments reuse them.

src/parameter.yml

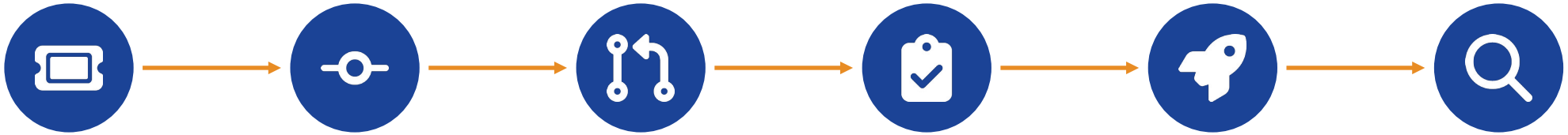
```yaml
find_replace:
  - find_value: "dev-sales…"
    replace_value:
      test: "test-sales…"
      prod: "prod-sales…"
```

In the pipeline

Azure Pipelines runs deploy.py as a service principal: Test first, then Prod behind an environment approval.

@ 2026 Dynamic Communities

The audit trail, end to end



Work item

The request and approval to make the change

Commit

What changed, who changed it, and when

Pull request

Independent review and sign-off

Build validation

Automated quality evidence

Deployment

Who promoted what, where, and when

Activity log

Publish and access events in production

Every link is captured while people do the work.

Nobody writes a change log after the fact. Check retention: keep pipeline runs, pull requests, and activity logs for as long as your audit window.

@ 2026 Dynamic Communities

Mapping it to change controls

COMMUNITY SUMMIT

Control	Evidence from this process
Changes are authorized	Every pull request requires a linked work item
Someone other than the author reviews	Branch policy: a required reviewer, and authors can’t approve their own changes
Changes are tested before production	Build validation on the PR, then sign-off in Test
Only authorized people deploy to Prod	A Prod approval gate; a service principal deploys, not individuals
Every change is traceable	Commit history, PR records, and deployment history

Your auditors define the controls. Treat this as a starting map, not compliance advice.

@ 2026 Dynamic Communities

Where Terraform fits

CONTENT · CHANGES DAILY

Semantic models | Reports | Notebooks

Managed by: **Git + deployment pipelines or fabric-cicd**

PLATFORM · CHANGES RARELY

Workspaces | Capacity assignment

Role assignments | Git connections

Managed by: **Terraform (microsoft/fabric provider)**

Reach for Terraform when…

- You create workspaces for every project and environment
- Roles and naming must match everywhere
- You want infrastructure changes reviewed as code, with drift caught by terraform plan

Skip it when…

- You have a handful of workspaces and one small team

@ 2026 Dynamic Communities

Terraform for Fabric: a first look

COMMUNITY SUMMIT

```
terraform/main.tf (condensed)

provider "fabric" {}  # Azure CLI sign-in or a service principal

resource "fabric_workspace" "env" {
  for_each     = toset(["Dev", "Test", "Prod"])
  display_name = "Sales [${each.key}]"
  capacity_id  = data.fabric_capacity.this.id
}

resource "fabric_workspace_role_assignment" "developers" {
  for_each     = fabric_workspace.env
  workspace_id = each.value.id
  principal    = { id = var.developer_group_id, type = "Group" }
  role         = each.key == "Dev" ? "Contributor" : "Viewer"
}

resource "fabric_workspace_git" "dev" {
  workspace_id            = fabric_workspace.env["Dev"].id
  initialization_strategy = "PreferRemote"
  git_credentials         = { source = "Automatic" }
  git_provider_details    = { git_provider_type = "AzureDevOps", … }
}
```

`terraform plan`

Shows exactly what will change, and catches drift, before anything changes

Same code, every project

A new project is a new tfvars file

Know the limits

Not every Fabric resource supports service principals yet. Check each resource's docs.

@ 2026 Dynamic Communities

Key takeaways

1. **Right-size first**

   Match the rigor to the risk. Not every report needs a pipeline.

2. **Start with PBIP and TMDL**

   Once Power BI is text, you can diff it, review it, and merge it.

3. **Let guardrails do the enforcing**

   Branch policies and build validation beat good intentions.

4. **Pick one promotion path and automate it**

   Deployment pipelines or fabric-cicd, with an approval in front of Prod.

5. **Terraform the platform, Git the content**

   Use each tool for the layer it was built for.

@ 2026 Dynamic Communities

Objectives Recap

1. Design a practical CI/CD pipeline for Power BI and Fabric that incorporates branching strategies, deployment pipelines, and appropriate governance guardrails for your team's environment.

2. Explain how Git integration and TMDL work together to transform Power BI and Fabric assets into versionable, auditable code that supports change tracking, collaboration, and compliance requirements.

3. Evaluate where Terraform fits into a Git-based deployment framework, understanding how it extends your pipeline to automate workspace provisioning and Fabric object management at scale.

@ 2026 Dynamic Communities

The largest independent Microsoft AI & Business Applications User Conference on the planet.

COMMUNITY SUMMIT

Thank you for Attending!

Slides & code

LinkedIn

YouTube

Connect with me

LinkedIn: /in/michaellmckinley
X: @_mikemckinley_
Web: www.mckinley.consulting
YouTube: @thedatapractitioner

Stick around for Q&A

@ 2026 Dynamic Communities